PolyRMC — setting up a new machine

From a computer with nothing on it to a working installation, assuming this repository is all you have.

Commands are Windows PowerShell, because that is what the instrument machines run. Nothing here needs an API key until section 6, and most labs never get there: the measurement path — opening files, processing runs, making figures — runs with no model and no key at all.

1. Install Python

You need **Python 3.11 or newer**. Check whether it is already there:

```
py --version
```

If that prints a version of 3.11 or above, skip to section 2. Otherwise install it from **python.org/downloads** and, at the first screen of the installer, tick **"Add python.exe to PATH"**. Close and reopen PowerShell afterwards, then check `py --version` again.

The Microsoft Store build of Python also works, but installs into a sandboxed location that makes paths harder to reason about later. The python.org installer is the easier one to support.

2. Put the code somewhere

Copy or clone the repository to a path **without spaces or OneDrive in it**. `C:\lab\polyrmc` is a good choice; `C:\Users\you\OneDrive\My Documents\` is not — OneDrive syncs files out from under a running process, and spaces make every later command need quoting.

```
cd C:\lab\polyrmc
dir
```

You should see `pyproject.toml`, `src`, `tests`, `docs` and `examples`. If `pyproject.toml` is missing you are in the wrong folder, or one level too high.

3. Create the environment and install

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -e ".[test]"
```

This needs a working internet connection: it fetches numpy, scipy, pandas, pydantic, matplotlib and python-dotenv from PyPI, plus pytest so that section 4 can verify the result. It installs **no model libraries and no vendor SDKs** — those are optional and covered in section 6.

Plain `pip install -e .` also works and gives you a working installation; it just leaves out pytest, so the check in the next section would have nothing to run.

Note the `.\.venv\Scripts\python.exe` prefix. A bare `python` is usually not on PATH on these machines, and activating the environment is one more thing to forget between sessions. Every command in this guide uses the full path, so it works from a cold terminal.

Ignore `requirements.txt` if you find it. It predates the packaging in `pyproject.toml`, lists the orchestration libraries and all three vendor SDKs as though they were required, and installing from it pulls in SDKs for providers you will never key. `pip install -e .` is the supported route.

4. Check the installation

```
.\.venv\Scripts\python.exe -m pytest
```

This is the verification. Just under five hundred tests, needing no API key, no network and no instrument file — they generate their own synthetic acquisitions with known answers, so a pass means the arithmetic on this machine is right, not merely that the imports resolved. It takes several minutes, mostly rendering real figures.

Then check the command itself responds:

```
.\.venv\Scripts\python.exe -m polyrmc.cli plot
```

It should answer `plot needs --file <path>`. A readable sentence rather than a traceback means the entry point is wired up.

There is no sample data, and that is deliberate

A fresh copy of this repository contains **no instrument files and no processed runs**. Everything under `data/` is excluded, because traces and sidecars are lab data and this repository is public. So there is nothing in the box to open, and the first real file you point it at will be one of your own.

5. Open your first file

Point it at any data file with no other arguments and it describes what is inside:

```
.\.venv\Scripts\python.exe -m polyrmc.cli plot --file "D:\data\run.txt"
```

It reads ARGEN acquisitions, KaleidaGraph `.QDA` files, processed CSVs this pipeline wrote, and ordinary delimited exports. It reports which kind it decided the file was, which columns can go on an axis, and whether it had to guess at the format.

Then draw something:

```
.\.venv\Scripts\python.exe -m polyrmc.cli plot `
    --file "D:\data\run.txt" --x seconds --y ls8
```

The PNG lands beside the data file. `--out` puts it elsewhere, `--style scatter` draws points, `--logy` gives a log axis. Column names containing spaces need quotes; if you get one wrong, the error lists the real ones.

Figures made this way carry an **exploratory** stamp in the corner, because you chose those columns by hand and nothing was processed or reviewed. Figures the pipeline produces are not stamped, because they were.

6. Optional: the agent session

Only needed to *talk* to the system. Install the SDK for the provider you actually buy tokens from — there is no reason to install one for a vendor you will never key:

```
.\.venv\Scripts\python.exe -m pip install -e ".[gemini]"
.\.venv\Scripts\python.exe -m polyrmc.cli setup
```

Substitute `[claude]`, `[openai]`, or `[all]`. `polyrmc setup` lists the providers, shows which are keyed, gives you the URL to buy a key, and spends one tiny call confirming the key works — better to learn that now than three questions into a session. The key is stored in `.env` at the project root, which is gitignored; setup prints the exact path.

Provider	Environment variable	Where to buy
gemini	`GEMINI_API_KEY` or `GOOGLE_API_KEY`	aistudio.google.com/apikey

claude	`ANTHROPIC_API_KEY`	console.anthropic.com/settings/keys
openai	`OPENAI_API_KEY`	platform.openai.com/api-keys

These are **API keys, billed per token**. A ChatGPT Plus or Claude Pro subscription is a separate product and does not grant API access.

```
.\.venv\Scripts\python.exe -m polyrmc.cli agent
```

Every turn prints what it spent; `/cost` gives the session total. Anything that would **change** a run stops and asks you first. Reading and computing do not, because approving every read only teaches you to click yes without looking.

7. Where files live

Everything hangs off the project root — the folder holding `pyproject.toml` — whichever directory you launch from. So you can run these commands while sitting in the folder with your raw files and it will still find the run history and your saved key.

To keep data somewhere else entirely, on a data drive or a share:

```
$env:POLYRMC_ROOT = "D:\lab\polyrmc-data"
```

Set it in the same terminal before running, or add it to the machine's environment variables to make it permanent.

8. A machine with no internet

The install in section 3 is the only step that needs the network. On a machine that cannot reach PyPI, fetch the wheels on one that can — matching the target's Python version and Windows architecture — carry them across, and install from the folder:

```
# on a connected machine
py -m pip download numpy scipy pandas pydantic matplotlib python-dotenv pytest -d wheels

# on the instrument machine, with wheels\ copied alongside
.\.venv\Scripts\python.exe -m pip install --no-index --find-links wheels -e .
```

Nothing else in the system reaches the network unless you key a provider and start a session.

Troubleshooting

Symptom	What it means
`py` is not recognised	Python is not installed, or was installed without “Add to PATH”. Reopen PowerShell first; if it still fails, re-run the installer and choose Modify.
`python` is not recognised	Expected. Use `.\.venv\Scripts\python.exe`, not a bare `python`.
`No module named polyrmc`	The install in section 3 did not run, or ran against a different environment. Re-run it from the folder containing `pyproject.toml`.
Running scripts is disabled on this system	You tried to activate the environment. You do not need to — call `.\.venv\Scripts\python.exe` directly.

“could not be read as a table”	The file has no header row, or its numbers are quoted. Open the first two lines in a text editor.
A column name is rejected	The message lists the real names. In an ARGEN file the instrument's own time column is text; the numeric one is `seconds`.
A `.QDA` column reads as empty	Read the notes printed underneath. A column stored as text is reported by name rather than silently skipped.
Somebody asks for a `.QDR`	No such format exists. KaleidaGraph writes `.QDA` for data and `.QPC` for a graph; they mean one of those.
A provider failed	The message names the category. Out of quota is a billing matter; a rejected key wants `polyrmc setup`; a stale model id wants `--model`. It will not quietly fail over onto another of your keys, because that is your money.

What to read next

`docs/polyrmc-architecture.pdf` — what the system is, what each tier may and may not do, the reverse-engineered KaleidaGraph formats, and the known gaps.

`docs/LAB-SETUP.md` — the same ground as this document, browsable in a text editor or on the repository page.

Both this guide and the architecture document are generated from the code they describe. After changing anything they cover, rebuild:

```
.\.venv\Scripts\python.exe scripts\build_setup_pdf.py
.\.venv\Scripts\python.exe scripts\build_architecture_pdf.py
```